RAG-Based Customer Support Assistant

with LangGraph & Human-in-the-Loop

LOW LEVEL DESIGN DOCUMENT

Internship Project | Batch: adv-genai-feb-2026

Submitted by: Ms. Tanishka Prasad Jadhav
Intern ID: IN226010302

Table of Contents

Sr. No.	Section / Subsection	Page No.
1	Module-Level Design	1
1.1	Document Processing Module	1
1.2	Chunking Module	1
1.3	Embedding Module	1
1.4	Vector Storage Module	1
1.5	Retrieval Module	1
1.6	Query Processing Module	1
1.7	Graph Execution Module	1
1.8	HITL Module	1
2	Data Structures	1
2.1	Document Representation	1
2.2	Chunk Format	1
2.3	Embedding Structure	1
2.4	Query-Response Schema	1
2.5	Graph State Object	1
3	Workflow Design (LangGraph)	1
3.1	Node Definitions	1
3.1.1	Node 1: process	1
3.1.2	Node 2: answer	1
3.1.3	Node 3: hitl	1
3.2	Graph Construction	1
3.3	Graph Flow Diagram	1
4	Conditional Routing Logic	1
4.1	Routing Function	1

4.2	Answer Generation Criteria	1
4.3	Escalation Criteria	1
4.4	Future Routing Extensions	1
5	HITL Design	1
5.1	When Escalation Is Triggered	1
5.2	What Happens After Escalation	1
5.3	How Human Response Is Integrated (Future Design)	1
6	API / Interface Design	1
6.1	Input Format	1
6.2	Graph Invocation Interface	1
6.3	Streamlit UI Interaction Flow	1
6.4	Output Format	1
7	Error Handling	1
7.1	Missing or Corrupt PDF	1
7.2	No Relevant Chunks Found	1
7.3	LLM Failure (Groq API Error)	1
7.4	Error Handling Summary	1

1. Module-Level Design

The system is composed of seven distinct modules, each with a clear responsibility boundary. Together they form a pipeline from raw PDF to LLM-generated answer.

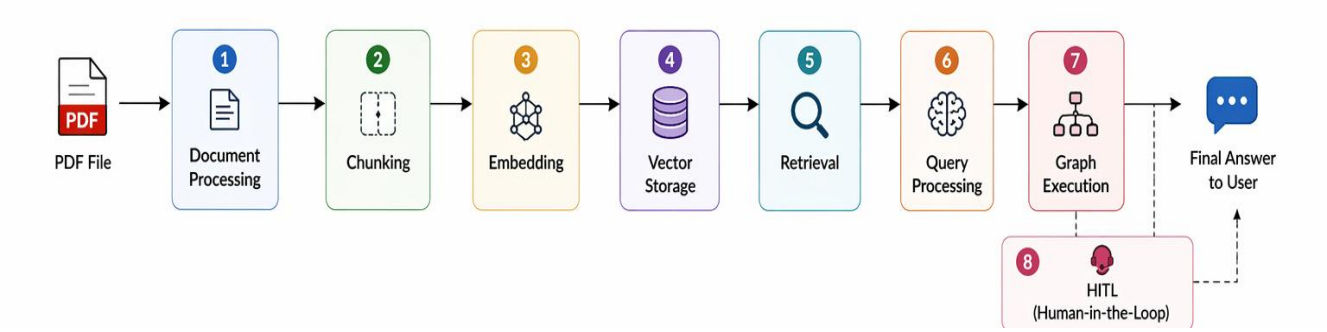


Fig 1.1: Module level design

1.1 Document Processing Module

File	backend/pipeline.py
Function	load_pdf(file_path: str) -> str
Input	Absolute file path to a PDF file
Output	Concatenated text from all PDF pages (str)
Library Used	pypdf.PdfReader
Error Handling	Skips pages where page.extract_text() returns None

```
def load_pdf(file_path):
    reader = PdfReader(file_path)
    text = ""
    for page in reader.pages:
        page_text = page.extract_text()
        if page_text:            # guard against blank/image-only pages
            text += page_text
    return text
```

1.2 Chunking Module

File	backend/pipeline.py
Function	chunk_text(text, chunk_size=500, overlap=50) -> List[str]
Strategy	Fixed-size sliding window with character overlap
Chunk Size	500 characters
Overlap	50 characters
Why Overlap	Prevents information loss at chunk boundaries

```
def chunk_text(text, chunk_size=500, overlap=50):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunk = text[start:end]
        chunks.append(chunk)
        start += (chunk_size - overlap)   # sliding window step
    return chunks
```

1.3 Embedding Module

File	backend/pipeline.py
Function	create_vector_store(chunks: List[str]) -> Chroma
Current Embedding	FakeEmbeddings(size=384) — random vectors for dev/test
Production Embedding	HuggingFaceEmbeddings('all-MiniLM-L6-v2')
Output	Chroma vectorstore instance with embedded chunks

```
def create_vector_store(chunks):
    embedding = FakeEmbeddings(size=384)    # replace with real embeddings in prod
    vectordb = Chroma.from_texts(
        texts=chunks,
        embedding=embedding
    )
    return vectordb
```

1.4 Vector Storage Module

Technology	ChromaDB (via langchain_community)
Mode	In-memory (current); persistent via persist_directory param
Index Type	HNSW (Hierarchical Navigable Small World) by default
Collection	Default unnamed collection per session
Upgrade Path	Chroma(persist_directory='./chroma_db', embedding_function=...)

1.5 Retrieval Module

File	backend/pipeline.py
Function	get_retriever(vectordb) -> VectorStoreRetriever
Method	vectordb.as_retriever(search_kwargs={'k': 3})
Search Type	Cosine similarity (approximate nearest neighbor)
Return	Top-3 Document objects with page_content field

```
def get_retriever(vectordb):
    return vectordb.as_retriever(search_kwargs={"k": 3})
```

1.6 Query Processing Module

File	backend/pipeline.py
Function	generate_answer(context: str, question: str) -> str
LLM Provider	Groq API — groq.Groq client
Model	llama-3.1-8b-instant
Temperature	0.2
Max Tokens	500
Prompt Format	System rules + Context block + Question

```
def generate_answer(context, question):
    prompt = f"""
You are a professional customer support assistant.
STRICT RULES:
- Answer ONLY using the provided context
- Be clear, structured, and helpful
- Use bullet points if needed
- Do NOT sound robotic
IF answer is NOT in context:
Say: "The document does not contain enough information..."

Context:
{context}

Question:
{question}

Answer:
"""
    response = client.chat.completions.create(
        model=MODEL_NAME,
        messages=[{"role": "user", "content": prompt}],
        temperature=0.2,
        max_tokens=500
    )
    return response.choices[0].message.content
```

1.7 Graph Execution Module

File	backend/graph.py
Function	build_graph(retriever) -> CompiledGraph
Framework	LangGraph — StateGraph(dict)
Nodes	process, answer, hitl
Entry Point	set_entry_point('process')
Finish Points	set_finish_point('answer'), set_finish_point('hitl')

1.8 HITL Module

File	backend/graph.py
Node	hitl(state) -> dict
Trigger	state['route'] == 'HITL' (context < 50 chars)
Current Action	Sets state['response'] = 'Escalated to human support.'
Future Action	Create support ticket via Zendesk API, send email alert

2. Data Structures

2.1 Document Representation

Raw text extracted from PDF is a single Python string. Each page's text is concatenated sequentially.

```
# Type: str
raw_text: str = "Full text content from all PDF pages concatenated..."
# Example: "Company Policy Manual v2.0\n\nSection 1: Return Policy\n..."
```

2.2 Chunk Format

After chunking, each piece of text is stored as a plain Python string in a list. LangChain wraps these in Document objects internally.

```
# Type: List[str]
chunks: List[str] = [
    "Company Policy Manual v2.0\n\nSection 1: Return Policy\nCustomers may return...",
    "...return items within 30 days. Items must be unused. Original receipt required...",
    # ...more chunks
]
```

2.3 Embedding Structure

```
# After embedding, each chunk maps to a float vector
# FakeEmbeddings(size=384) generates random 384-dimensional vectors

# Internally stored in ChromaDB as:
{
    "id": "uuid-string",
    "embedding": [0.123, -0.456, 0.789, ...],  # 384 floats
    "document": "Company Policy Manual v2.0...",
    "metadata": {}
}
```

2.4 Query-Response Schema

The retriever returns a list of LangChain Document objects. The final response is a plain string.

```
# Retriever returns:
docs: List[Document] = [
    Document(page_content="...relevant chunk 1...", metadata={}),
    Document(page_content="...relevant chunk 2...", metadata={}),
    Document(page_content="...relevant chunk 3...", metadata={}),
]

# Context assembled as:
context: str = " ".join([doc.page_content for doc in docs])

# Final response:
response: str = "Based on the document, returns are accepted within 30 days..."
```

2.5 Graph State Object

LangGraph uses a Python dict as the state object. All nodes read from and write to this shared state.

```
# Type: TypedDict (effectively dict with defined keys)
state = {
    "question": str,    # User's query — set at graph invocation
    "context":  str,    # Retrieved chunks joined — set by process node
    "route":    str,    # "ANSWER" or "HITL" — set by process node
    "response": str,    # Final LLM/HITL response — set by answer/hitl node
}

# Initial invocation:
result = graph.invoke({"question": "What is the return policy?"})

# After graph completes:
# result["response"] = "Returns are accepted within 30 days of purchase..."
```

3. Workflow Design (LangGraph)

The LangGraph StateGraph defines a directed graph of processing nodes. Each node is a Python function that reads from and writes to the shared state dict.

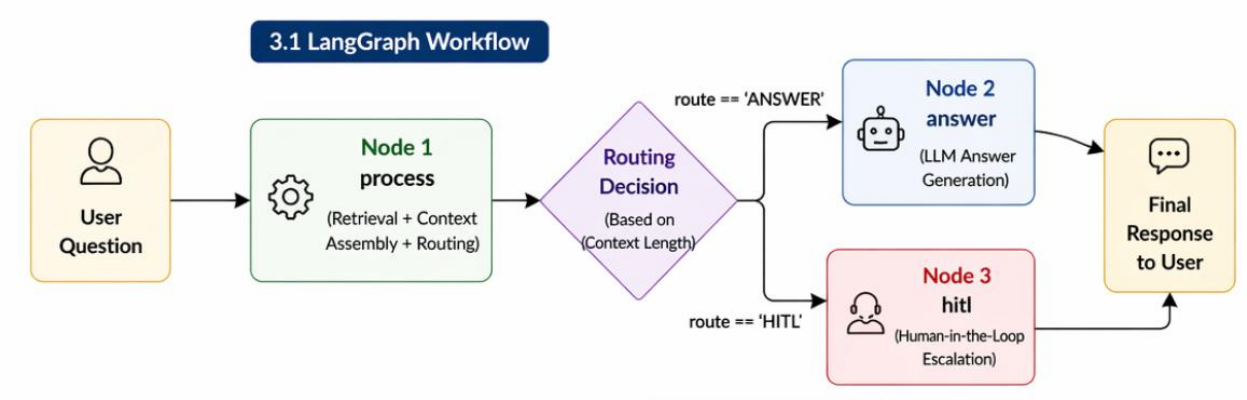


Fig 3.1 : Workflow design

3.1 Node Definitions

Node 1: process

Purpose	Core processing node — retrieval + context assembly + routing decision
Input State Keys	question
Output State Keys	context, route
Retriever Call	retriever.invoke(question) → top-3 Document objects
Context Assembly	Join page_content of all docs with space separator
Routing Logic	len(context.strip()) < 50 → 'HITL' else 'ANSWER'

```
def process(state):
    question = state["question"]
    docs = retriever.invoke(question)
    context = " ".join([
        doc.page_content if hasattr(doc, "page_content") else str(doc)
        for doc in docs
    ])
    state["context"] = context
    state["route"] = "HITL" if len(context.strip()) < 50 else "ANSWER"
    return state
```

Node 2: answer

Purpose	LLM answer generation — calls Groq API with context and question
Input State Keys	context, question
Output State Keys	response
Called When	route == 'ANSWER'

```
def answer(state):
    response = generate_answer(state["context"], state["question"])
    state["response"] = response
    return state
```

Node 3: hitl

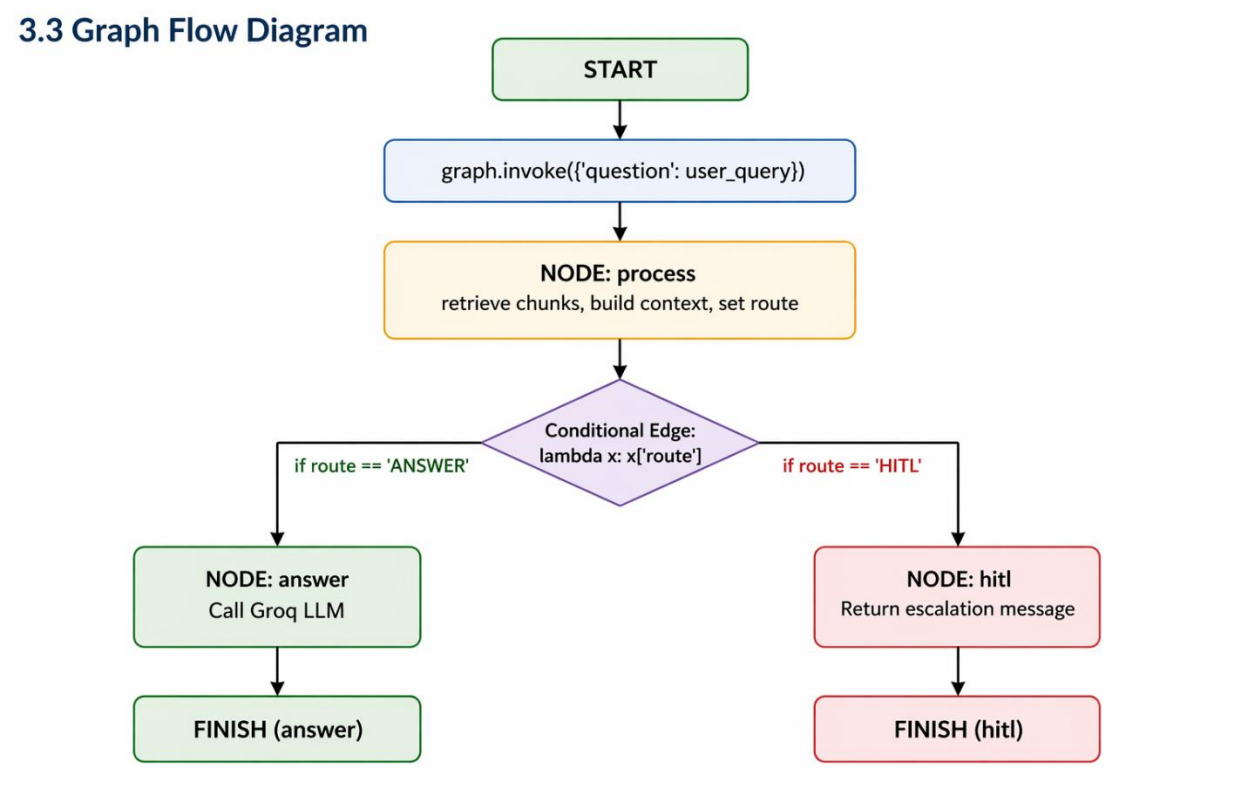
Purpose	Human-in-the-Loop escalation — returns escalation message
Input State Keys	context (insufficient), question
Output State Keys	response
Called When	route == 'HITL'

```
def hitl(state):
    state["response"] = "Escalated to human support."
    return state
```

3.2 Graph Construction

```
builder = StateGraph(dict)

builder.add_node("process", process)
builder.add_node("answer", answer)
builder.add_node("hitl", hitl)

builder.set_entry_point("process")

builder.add_conditional_edges(
    "process",                          # source node
    lambda x: x["route"],               # routing function
    {
        "ANSWER": "answer",             # route string → target node
        "HITL": "hitl"
    }
)

builder.set_finish_point("answer")
builder.set_finish_point("hitl")

graph = builder.compile()
```

3.3 Graph Flow Diagram



4. Conditional Routing Logic

4.1 Routing Function

The conditional routing is implemented as a lambda function passed to add_conditional_edges(). It reads the route key from state and maps it to a node name.

```
builder.add_conditional_edges(
    "process",
    lambda x: x["route"],      # reads state["route"]
    {
        "ANSWER": "answer",    # sufficient context → LLM answer
        "HITL": "hitl"         # insufficient context → human escalation
    }
)
```

4.2 Answer Generation Criteria

Condition	Value	Route	Reason
context length >= 50 chars	True	ANSWER	Sufficient content retrieved from vector DB
context has meaningful text	Implied	ANSWER	LLM has enough material to generate an answer
Groq API responds	True	ANSWER	Full answer lifecycle completes successfully

4.3 Escalation Criteria

Trigger Condition	Root Cause	Route	Action
context length < 50 chars	No matching chunks found in ChromaDB	HITL	Human escalation
All retrieved chunks are empty/whitespace	PDF had non-extractable content (images)	HITL	Human escalation
FakeEmbeddings fails to match semantically	Vectors are random (dev mode)	HITL	Human escalation
Query is in different language than document	Embedding mismatch	HITL	Human escalation
Query is ambiguous with no keywords	Poor retrieval signal	HITL	Human escalation

4.4 Future Routing Extensions

The routing logic can be extended to support more conditions using a dedicated routing function:

```
def route_query(state):
    context = state.get("context", "")
    question = state.get("question", "")

    if len(context.strip()) < 50:
        return "HITL"            # insufficient context

    low_confidence_keywords = ["i don't know", "cannot find", "no information"]
    if any(kw in context.lower() for kw in low_confidence_keywords):
        return "HITL"            # low confidence context

    if len(question.split()) > 50:
        return "HITL"            # overly complex query

    return "ANSWER"              # standard path
```

5. HITL Design

5.1 When Escalation Is Triggered

HITL (Human-in-the-Loop) escalation is triggered when the system cannot find sufficient relevant context to generate a reliable answer. The threshold is a context string of fewer than 50 characters after stripping whitespace.

Scenario	Context Length	Escalated?
Question matches document content well	> 50 chars	No — LLM answers
Question is completely off-topic	< 50 chars (near empty)	Yes — HITL
PDF contains only scanned images (no text)	0 chars	Yes — HITL
PDF uploaded but empty	0 chars	Yes — HITL

5.2 What Happens After Escalation

Current Implementation	state['response'] = 'Escalated to human support.' — message shown to user
Step 1 (Now)	System returns a polite escalation message in the chat interface
Step 2 (Planned)	Capture the unanswered query in a log file or SQLite DB
Step 3 (Planned)	Send email/Slack notification to support team with query details
Step 4 (Planned)	Create ticket in Zendesk/Freshdesk with conversation context
Step 5 (Planned)	Human agent responds via ticket system, response fed back to user

5.3 How Human Response Is Integrated (Future Design)

```
# Future HITL flow with ticket system

def hitl_advanced(state):
    question = state["question"]
    context_available = state.get("context", "None")

    # 1. Log the unanswered query
    ticket_id = create_ticket(
        subject=f"Unanswered query: {question[:60]}",
        description=f"Query: {question}\nContext found: {context_available}",
        priority="medium"
    )

    # 2. Notify human agent
    send_slack_alert(
        channel="#support-queue",
        message=f"New HITL escalation: Ticket #{ticket_id} | Query: {question}"
    )
```

```
    # 3. Return acknowledgment to user
    state["response"] = f"""Your query has been escalated to our support team.
Ticket ID: #{ticket_id}
Expected response time: 2-4 hours
You will be notified via email."""

    return state
```

6. API / Interface Design

6.1 Input Format

The system accepts two types of input from the user:

Input Type	Format	Handled By	Validation
PDF Document	Binary file upload (.pdf)	Streamlit st.file_uploader()	type=['pdf'] filter
User Query	Plain text string	Streamlit st.text_input()	query.strip() check

6.2 Graph Invocation Interface

```
# Graph input schema
graph_input = {
    "question": str    # Required — user's query string
}

# Graph invocation
result = graph.invoke(graph_input)

# Graph output schema
graph_output = {
    "question": str,    # Echoed from input
    "context":  str,    # Retrieved and joined chunk text
    "route":    str,    # "ANSWER" or "HITL"
    "response": str     # Final answer or escalation message
}
```

6.3 Streamlit UI Interaction Flow

Step 1	User opens Streamlit app at localhost:8501
Step 2	User uploads PDF via file_uploader widget
Step 3	App calls load_pdf() → chunk_text() → create_vector_store() → get_retriever()
Step 4	graph = build_graph(retriever) stored in st.session_state.graph
Step 5	User types query and clicks 'Ask' button
Step 6	result = graph.invoke({'question': query}) called
Step 7	result['response'] appended to st.session_state.chat_history
Step 8	Chat history rendered in reverse order (latest first)

6.4 Output Format

```
# Successful LLM answer
{
   "response": "Based on the document:\n• Return policy allows 30 days...\n• Items must be unused..."
}

# HITL escalation
{
   "response": "Escalated to human support."
}

# UI rendering (Streamlit)
st.markdown(f"""
<div class="response-box">
   <strong>Q: {chat['question']}</strong><br><br>
   <strong>A:</strong> {chat['response']}
</div>
""", unsafe_allow_html=True)
```

7. Error Handling

7.1 Missing or Corrupt PDF

```
# Current: No explicit try/except in load_pdf()
# Recommended addition:
def load_pdf(file_path):
    try:
        reader = PdfReader(file_path)
        text = ""
        for page in reader.pages:
            page_text = page.extract_text()
            if page_text:
                text += page_text
        if not text.strip():
            raise ValueError("PDF contains no extractable text (may be image-only)")
        return text
    except FileNotFoundError:
        raise FileNotFoundError(f"PDF file not found at: {file_path}")
    except Exception as e:
        raise RuntimeError(f"PDF processing failed: {str(e)}")
```

7.2 No Relevant Chunks Found

When the retriever finds no matching chunks (or only very short context), the routing system correctly escalates to HITL. This is the primary error-handling mechanism for retrieval failure.

```
# In graph.py process node:
context = " ".join([
    doc.page_content if hasattr(doc, "page_content") else str(doc)
    for doc in docs
])
# If docs is empty list → context = ""
# len("".strip()) = 0 < 50 → route = "HITL"  ✓
```

7.3 LLM Failure (Groq API Error)

```
# Recommended error handling for generate_answer():
def generate_answer(context, question):
    try:
        response = client.chat.completions.create(
            model=MODEL_NAME,
            messages=[{"role": "user", "content": prompt}],
            temperature=0.2,
            max_tokens=500
        )
        return response.choices[0].message.content
    except groq.APIConnectionError:
        return "Unable to connect to AI service. Please try again in a moment."
    except groq.RateLimitError:
        return "Service is currently busy. Please wait a moment and retry."
    except groq.AuthenticationError:
        return "API authentication failed. Please check your GROQ_API_KEY."
    except Exception as e:
        return f"An error occurred while generating the answer: {str(e)}"
```

7.4 Error Handling Summary

Error Type	Location	Current Handling	Recommended Handling
PDF not found	load_pdf()	Unhandled exception	try/except FileNotFoundError
Image-only PDF	load_pdf()	Empty string returned	Raise ValueError, show UI warning
No chunks found	process node	Route = HITL (correct)	Already handled
Groq API failure	generate_answer()	Unhandled exception	try/except with user message
Empty query	app.py	Partial — query.strip() check	Also check minimum length
GROQ_API_KEY missing	config.py	None value passed to Groq	Check env var at startup